SMART BICYCLE BRAKE LIGHT (SBBL)

IMAN MARIA KAIZA

Abstract

Cycling has become one of the most sustainable forms of mobility and transportation; however, cyclists remain vulnerable to rear-end collisions due to poor visibility and signalling of their braking actions. Despite the strong illumination from conventional rear lights, they fail to communicate braking activity and require cyclists to operate them manually. Moreover, existing brake light solutions have limitations, such as delayed reactions and false positives, lack of adaptive brightness, and limited compatibility with various bicycle models. This project aimed to design, develop, and simulate a Smart Bicycle Brake Light (SBBL) system using Tinkercad as its simulation environment. The system integrated brake detection, motion state recognition, manual OFF override, ambient-light brightness adaptability using an LDR and PWM, power management using well-defined sleep-mode programming, and sensor fault detection with a safety light brightness fallback. It was developed using a five-phase modular approach, with each phase tested separately before integration. The system operated across four modes: CYCLING, BRAKING, STATIONARY, and DEACTIVATED, using the non-blocking millis() function to continuously monitor input without delay. Testing confirmed that all the system requirements were met: testing methods included visual observation of the changes in the simulation as well as the review of Serial Monitor output to validate specific requirements, including power management and fault detection and safety fallback logic, that could not be physically tested in the simulation environment. Finally, the system's control logic, state machine, and fail-safe mechanisms were all validated, and physical prototype designs and future enhancements for the SBBL system were defined.

Table of Contents

Abstract 1

List of Figures 5

List of Tables 8

List of Abbreviations 9

Chapter 1: Introduction 10

1.1 Background 10

1.2 Problem Statement 11

1.3 Aim/ General Objective 12

1.4 Specific Objectives 12

1.5 Justification 13

1.6 Scope 14

1.7 Limitations and Delimitations 15

1.7.1 Limitations 15

1.7.2 Delimitations 15

Chapter 2: Review of Existing Solutions, and System Requirements and Design 17

2.1 Introduction 17

2.2 Review of Existing Systems 17

2.2.1 Review of The Lezyne Strip Drive Pro 400+ 17

2.2.2 Review of The Magene L508 Radar Tail Light 19

2.2.3 Review of the Sigma Sport Brake Light 21

2.3 System Requirements 23

2.3.1 Functional Requirements 23

2.3.2 Non-Functional Requirements 24

2.4 System Architecture 25

2.4.1 Block Diagram 27
2.4.2 State-Machine Logic 27
2.4.3 State Transition Diagram 29
2.4.4 State Transition Table 29
Chapter 3: Methodology and Simulation Implementation 31
3.1 Introduction 31
3.2 Development Methodology 31
3.3 Simulation Environment and Component Inventory 31
3.3.1 Tinkercad Circuits 31
3.3.2 Component Inventory 31
3.4 Phase Evaluations 33
3.4.1 Phase 1 Implementation: Core Brake Detection 33
3.4.2 Phase 2 Implementation: Motion Recognition and Manual Override 35
3.4.3 Phase 3 Implementation: Intelligent Lighting Management 37
3.4.4 Phase 4 Implementation: Power Management and Sleep Mode 39
3.4.5 Phase 5 Implementation: LDR Fault Detection and Safety Fallback 43
3.5 Final System Overview and Challenges 45
Chapter 4: Testing, Evaluation, and Simulation Limitations 47
4.1 Testing Methodology 47
4.2 Evaluation Against the Design Specifications 47
4.3 Functional and Non-Functional Testing Results 48
4.3.1 Functional Testing 48
4.3.2 Non-Functional Testing 52
4.4 Simulation Against Reality 52
4.4.1 Simulation Limitations 52

Chapter 5: Conclusion and Real-World Implementation 54
5.1 Introduction 54
5.2 Project Conclusion 54
5.3 Envisioned Physical Product Design 54
5.4 Product Models 55
5.4.1 The Permanent (Pre-Installed) Model 55
5.4.2 The Detachable (After-Market) Model 55
5.5 Recommendations And Future Enhancements 56
References 57

List of Figures

Figure 2.1 The Lezyne Strip Drive Pro 400+ (BikeRadar, n.d.).....17
Figure 2.2 The Magene L508 Radar Tail Light (Magene, n.d.) 19
Figure 2.3 The Sigma Sport Brake Light (Stead, 2015).....21
Figure 2.4 SBBL system architecture block diagram showing the input, processing, and output stages. 27
Figure 2.5 State transition diagram showing the four operating modes (CYCLING, BRAKING, STATIONARY, DEACTIVATED) and the triggers that cause the transitions between them.....29

Figure 3.1 SBBL Circuit Components 32
Figure 3.1 SBBL Circuit Components 32
Figure 3.2 Phase 1 circuit showing the dual brake buttons and the three red LEDs 33
Figure 3.2 Phase 1 circuit showing the dual brake buttons and the three red LEDs 33
Figure 3.3 Code Excerpt: The foundational code used to read the brake inputs and illuminate the LEDs.....34
Figure 3.3 Code Excerpt: The foundational code used to read the brake inputs and illuminate the LEDs.....34
Figure 3.4 Phase 2 circuit adding the motion slide switch and manual override button.....36
Figure 3.4 Phase 2 circuit adding the motion slide switch and manual override button.....36
Figure 3.5 Code Excerpt: The logic added to handle the stationary illumination hold and the manual override without blocking the loop.....37
Figure 3.5 Code Excerpt: The logic added to handle the stationary illumination hold and the manual override without blocking the loop.....37
Figure 3.6 Phase 3 circuit introducing the LDR for ambient-light sensing. 38
Figure 3.6 Phase 3 circuit introducing the LDR for ambient-light sensing. 38
Figure 3.7 Code Excerpt: The code added to classify the LDR reading into daylight, dusk, or nighttime brightness levels.....39
Figure 3.7 Code Excerpt: The code added to classify the LDR reading into daylight, dusk, or nighttime brightness levels.....39

Figure 3.8 Code Excerpt: The code used to apply the calculated brightness to the LEDs using analogWrite() only when the lightOn state machine logic is true. 39
Figure 3.8 Code Excerpt: The code used to apply the calculated brightness to the LEDs using analogWrite() only when the lightOn state machine logic is true. 39
Figure 3.9 Code Excerpt: The configuration of the hardware interrupts on pins D2 and D3, with their Interrupt Service Routines (ISRs) used to wake up the Arduino when a wake-up event is triggered. 41
Figure 3.9 Code Excerpt: The configuration of the hardware interrupts on pins D2 and D3, with their Interrupt Service Routines (ISRs) used to wake up the Arduino when a wake-up event is triggered. 41
Figure 3.10 Code Excerpt: The power-down sleep sequence: it highlights the safety checks and core commands used to stop and resume the microcontroller. 42
Figure 3.10 Code Excerpt: The power-down sleep sequence: it highlights the safety checks and core commands used to stop and resume the microcontroller. 42
Figure 3.11 Code Excerpt: At the end of the main loop, the function call evaluates the current system state and triggers sleep mode if the conditions are safe.....42
Figure 3.11 Code Excerpt: At the end of the main loop, the function call evaluates the current system state and triggers sleep mode if the conditions are safe.....42
Figure 3.12 Code Excerpt: The final safety logic ensuring maximum brightness during LDR sensor failure.....44
Figure 3.12 Code Excerpt: The final safety logic ensuring maximum brightness during LDR sensor failure.....44
Figure 3.13 Code Excerpt: The Serial Monitor verifies that the system detects an abnormal LDR reading and triggers the fallback brightness. 45
Figure 3.13 Code Excerpt: The Serial Monitor verifies that the system detects an abnormal LDR reading and triggers the fallback brightness. 45
Figure 3.14 The complete and fully integrated SBBL simulation circuit. 46

Figure 4.1 Visual comparison of the LED brightness levels. Note: Tinkercad's simulation limits the visual distinction between the pulse-width modulation (PWM) values, but the illuminations correspond to the following PWM values: Nighttime (255), Dusk (218), Daylight (180).....50

Figure 4.2 Code Excerpt: Serial Monitor output verifying the successful execution of the sleep mode and the wake-up interrupts. 51

List of Tables

Table 2.1 State transition rules for the SBBL state machine. 29

Table 3.1 Component inventory for the SBBL simulation, detailing their real-world equivalents and roles in the system. 32

Table 4.1 Component inventory for the SBBL simulation, detailing their real-world equivalents and roles in the system. 47
Table 4.2 Phase 1 Test Case 48
Table 4.3 Phase 2 Test Case 48
Table 4.4 Phase 3 Test Case 49
Table 4.5 Phase 4 Test Case 50
Table 4.6 Phase 5 Test Case 51
Table 4.7 Non-Functional Requirements Test Case 52

List of Abbreviations

ANT+ – Advanced Network Technology Plus

AVR – Alf and Vegard's RISC Processor

CPU – Central Processing Unit

GND – Ground

GPS – Global Positioning System

ISR – Interrupt Service Routine

LDR – Light Dependent Resistor

LED – Light Emitting Diode

mAh – Milliampere-hour

PCB – Printed Circuit Board

PWM – Pulse Width Modulation

SBBL – Smart Bicycle Brake Light

USB-C – Universal Serial Bus Type-C

Chapter 1: Introduction

1.1 Background

In recent years, cycling has experienced significant growth, transitioning from a recreational pastime into a major pillar of sustainable mobility. It provides a multitude of health, environmental, and economic advantages, including improving quality of life and increasing life expectancy, lowering fossil fuel use and greenhouse gas emissions, and reducing the financial burdens for individuals and economies with its low operational and infrastructural costs, respectively (Buehler & Goel, 2022).

Despite these advantages, cyclists face numerous safety issues, particularly when sharing the road with larger and faster vehicles. With relatively little protection, cyclists are more vulnerable during collisions and accidents. Their smaller size, limited lighting and signalling equipment make them less apparent to other road users, especially in conditions involving poor visibility.

Most notably, their safety extends beyond collisions involving overtaking vehicles. Cyclists tend to experience rear-end collisions because other road users fail to recognise changes in their movement or are unaware of how to respond appropriately in their presence. The United Kingdom, for example, has identified rear-end collisions as an important category of crashes involving cyclists, noting that low visibility is a significant risk factor associated with rear-end crashes, highlighting the importance of visibility in cycling safety (Chao, et al., 2025).

Visibility becomes highly important when a cyclist slows down, applies their brakes, or comes to a complete stop. In such situations, other road users travelling behind are left trying to recognise changes in the bicycle's movement and interpret the cyclist's actions and hand signals. These methods are insufficient, especially in situations with limited visibility, distractions or varying road conditions. Therefore, lighting systems provide an additional means of communicating a cyclist's presence, actions, and movements to nearby road users.

Many cyclists use rear lights to improve their bicycle's visibility. Different illumination modes make them noticeable to other road users; however, their primary function is generally to illuminate rather than explicitly communicate the cyclist's actions. Consequently, they don't provide specific indications that the cyclist is slowing down or preparing to stop.

This distinction highlights the need for brake-activated lighting systems. Unlike conventional rear lights that remain illuminated throughout, brake-activated systems respond to braking by illuminating when the cyclist slows down or stops. This provides other road users with a clear visual indication of the cyclist's braking actions, allowing them to properly anticipate changes in speed, maintain safe distances, and react appropriately, thereby reducing the risk of rear-end collisions.

Existing research demonstrates that the enhancement of bicycle lighting systems using automated lighting, electronic sensors, and a microcontroller can improve cyclists' communication and safety (Liao, et al., 2025). For example, Hamilton and Orsi (2013) developed a bicycle turn and brake system that detected changes in cycling behaviour and automatically activated corresponding signals when turning, slowing down, or braking (Hamilton & Orsi, 2013).

Beyond detecting braking activity, intelligent bicycle lighting systems should incorporate features and functions to improve visibility and responsiveness, including recognising movement and stationary conditions, maintaining illumination once the bicycle stops, and adjusting light intensity according to the surrounding conditions. Such features help ensure continued visibility and appropriate illumination in both light and dark settings. These factors provide the basis for the Smart Bicycle Brake Light (SBBL) system, which proposes an intelligent lighting system that integrates mechanical brake detection, motion-state recognition, stationary illumination, manual override functionality and surrounding-based brightness adjustment to address the gaps that have been overlooked.

1.2 Problem Statement

Conventional bicycle rear lights primarily improve visibility with their continuously illuminated lights and predetermined flashing patterns; however, they fail to provide a distinct indication of braking activity. Consequently, nearby road users have to observe the cyclist's physical movements and speed changes to recognise when they are slowing down. This can delay their response, potentially leading to misinterpretation of cyclists and causing rear-end crashes.

It becomes more difficult to recognise cyclists' actions in low visibility, with researchers noting that it causes more rear-end collisions, highlighting the importance of visibility in addressing cycling safety (Chao, et al., 2025). Additionally, most bicycle lighting systems depend on cyclists manually switching their lights on or off or selecting their preferred modes. This is very

inconvenient because cyclists must remember to activate their lights and operate them while concentrating on riding and navigating the road. Additionally, lighting systems with a fixed brightness cannot adapt to changing environmental conditions, creating the need for systems that can maintain appropriate illumination while adjusting brightness according to the light intensity and levels in the cyclist's environment.

Existing solutions showcase the potential of integrating electronic sensors and microcontrollers to develop automated bicycle systems. For example, some systems use pressure sensors integrated into the brake lever to detect braking and activate a rear light (Liao, et al., 2025). Similarly, Hamilton and Orsi (2013) explored sensor-based detection of bicycle behaviour to activate corresponding signals (Hamilton & Orsi, 2013). These developments provide a foundation for investigating more integrated light systems that can combine braking detection with additional functionalities.

Therefore, the problem addressed by this project is the limited ability of conventional bicycle rear lights to automatically communicate braking activity while accommodating stationary conditions and changing environmental lighting. The Smart Bicycle Brake Light (SBBL) system proposes an automatically controlled system that uses mechanical brake detection, recognises simulated movement and stationary conditions, maintains illumination after the bicycle stops, and provides a manual OFF override when stationary. Additionally, an ambient light sensor adjusts LED brightness based on the surrounding light. This system is developed and evaluated in Tinkercad to demonstrate how these functions can be integrated into a fully functional, coordinated bicycle brake light system before potential physical implementation.

1.3 Aim/ General Objective

To design, develop, and simulate a smart bicycle brake light system that automatically detects braking activity, recognises moving and stationary states, maintains illumination when braking and stationary, provides a manual OFF override, and adjusts LED brightness according to the light conditions of the surrounding environment.

1.4 Specific Objectives

i. To evaluate existing bicycle brake light solutions and identify how SBBL addresses their limitations.
ii. To determine the functional and non-functional requirements of the SBBL system.

iii. To design the SBBL electronic circuit, system architecture, and state machine control logic.

iv. To develop a Tinkercad simulation integrating brake detection, motion state recognition, stationary illumination, manual OFF control and override, and ambient-light brightness adjustment.

v. To test the developed SBBL simulation against the system requirements and evaluate its performance within the constraints and limitations of Tinkercad.

1.5 Justification

Conventional rear lights illuminate cyclists, but they don't indicate a cyclist's braking activity. Additionally, they depend on the cyclist manually switching them on and off and selecting modes of illumination whilst concentrating on the road (Pai, 2011). Thus, this project develops an extensive solution to address the critical problems in conventional bicycle rear lights.

Creating an automated bicycle brake light system with brake detection, stationary illumination, and ambient-light adjustment will enhance road safety by providing clear visual signals to other road users, allowing them to accurately interpret speed changes and maintain a safe distance from cyclists (Buehler & Goel, 2022). Dual brake lever triggers will automatically activate the rear LEDs, ensuring expeditious communication of deceleration and reducing rear-end collision risks (Pai, 2011). Moreover, the maintenance of stationary illumination keeps the brake light active when stopping at junctions or traffic lights, while a manual override gives the cyclist control to switch off the light when done, preventing unintended reactivation until movement resumes.

By integrating these features in a state machine framework, the system is able to coordinate input signals, including motion state and ambient brightness, to adjust the illumination adaptively without needing the cyclist to manually operate the system. Automatically tailoring the LED's brightness to the surrounding light levels provides optimal visibility in varying weather and lighting environments without overriding the original braking functions. Ultimately, this solution frees the rider from any manual distraction while providing clear, dynamic signals that allow nearby road users to react effectively and expeditiously, creating a safer environment for cyclists.

Before physical implementation, a simulation can evaluate, test, and refine these features. Simulation allows circuit connections, component interactions, and program behaviour to be reviewed. It recreates different operating conditions, making it possible to observe how the system

responds to braking, motion state, manual override, and surrounding light changes. Modular development allows individual functionality to be tested and revised before integration, and although the product is a simulation rather than a physical product, it establishes functions of an electronic design that could serve as a foundation for future physical development.

1.6 Scope

The proposed study primarily focuses on the design, development, and simulation of an intelligent bicycle brake light system using an Arduino Uno microcontroller, integrating brake detection, motion state recognition, stationary illumination, manual OFF override, ambient-light brightness adjustment, and potential LDR (Light Dependent Resistor) fault detection with a predetermined light brightness level to fall back on. This system aims to improve cyclists' communication and safety by automatically activating a rear brake light to indicate braking activity, maintaining illumination after the bicycle stops, allowing the cyclist to manually deactivate the stationary illumination, and adjusting LED brightness according to surrounding light conditions using an LDR to detect the light levels and pulse-width modulation (PWM) to control LED brightness. The project focuses on developing an electronic circuit, selecting components, implementing the control logic around four operating states, and testing the resulting simulation. The project will be developed and evaluated using Tinkercad Circuits as the simulation environment, with the following equipment: an Arduino Uno microcontroller, two mechanical pushbutton switches to simulate the left and right brake levers, a slide switch to simulate moving and stationary states, a pushbutton to represent the manual OFF override button, three LEDs with current-limiting resistors, and an LDR with a voltage-divider resistor to measure the ambient light. This project operates on the assumption that the cyclist has a front light for visibility at night and in dimly lit settings, meaning the rear unit functions as a brake light entirely rather than a continuously illuminated rear light. Moreover, this project is a simulation and not a physically developed, installed, or road-tested bicycle brake light product: it didn’t involve any physical installation of brake switches on the brake levers, actual bicycle motion detection, real-world road testing, measurement of visibility from different distances under real road conditions, physical verification of the ATmega328P’s (a microcontroller chip found in the Arduino Uno) power down and sleep mode, or any assessment for commercial use, and it did not include a continuous rear-light mode, ultrasonic rear-object or proximity detection, or any advanced features beyond the implemented brake light logic.

1.7 Limitations and Delimitations

1.7.1 Limitations

Although the smart bicycle brake light simulation demonstrates the integration of several features and functions, its reliance on Tinkercad presents certain limitations. Since the project will be developed entirely within a simulated environment, it cannot fully recreate the physical conditions of an actual bicycle brake lighting system, including environmental exposure, vibrations, mechanical wear, humidity, or temperature changes. Moreover, the use of electronic circuit equipment, like switches to simulate brake levers and a slide switch to simulate motion change, means that the system will not be tested with actual mechanical brake inputs or physical motion sensors. Similarly, the simulated LEDs and LDR demonstrate the control logic and brightness adjustment to environmental light, but they don't establish the real-world visibility, light distribution, or the accuracy of the sensors in the final product.

As a result, the project cannot verify the effectiveness of the system in communicating braking intentions to other road users under real road conditions, nor does it establish the suitability, durability, or long-term reliability of the selected electronic components for practical bicycle installation. Factors such as physical dimensions, battery life, electrical durability, and power usage would need further calibration and physical equipment testing. In the end, while the simulation confirms the programmed behaviour under predefined conditions, it doesn't assess the system's commercial usability or real-world effectiveness.

1.7.2 Delimitations

On the other hand, certain aspects were intentionally excluded to maintain a clear scope for development. The project is restricted to a simulated electronic prototype, excluding physical assembly, bicycle installation, and road testing. Moreover, mechanical brake switches were chosen over accelerometers because they have delayed reaction times, and their motion sensors often mistake rough gravel and potholes for braking, leading to false positives. Mechanical switches provide direct and accurate detection of the cyclist's braking activity, whereas accelerometers depend on inferred deceleration and often fail to register gradual stops. Furthermore, a slide switch will be used to simulate moving and stationary states instead of other complex physical sensors like wheel rotation or GPS. Finally, the ambient light sensor (LDR) is strictly limited to adjusting

the LED brightness when illumination is already needed; it doesn't independently activate the lights, ensuring the system's LEDs are purely controlled by the brakes and stationary periods.

The system focuses exclusively on brake-activated and stationary illumination, purposely excluding a continuous rear-light mode. It also doesn't incorporate an automatic shut-off timer for the stationary illumination; instead, the light will stay on until the cyclist either activates the manual OFF override or continues cycling. Finally, additional bicycle signalling features, such as turning indicators, hazard lights, wireless communications, and GPS tracking, are excluded to ensure that the development remains focused on the core combination of brake detection, motion-state recognition, and automatic ambient-aware brightness adjustment through simulation.

Chapter 2: Review of Existing Solutions, and System Requirements and Design

2.1 Introduction

This chapter reviews three existing bicycle brake-light solutions, including conventional rear bicycle lights, accelerometer-based bicycle brake lights, and mechanically activated brake lights, determining their limitations and establishing the need for a system that combines direct mechanical braking with motion state recognition and ambient light adjustment. Based on these gaps, the functional and non-functional requirements for the SBBL system are defined. Finally, the system architecture and state machine logic are designed and illustrated to outline their inputs, processes, and outputs, as well as their four distinct operating modes.

2.2 Review of Existing Systems

2.2.1 Review of The Lezyne Strip Drive Pro 400+



Figure 2.1 The Lezyne Strip Drive Pro 400+ (BikeRadar, n.d.)

The Lezyne Strip Drive Pro 400+ is a premium, high-performing conventional bicycle rear light. Designed with 10 LEDs and a maximum output of 400 lumens, it is explicitly engineered to

provide optimal light dispersion and extreme visibility in even the harshest environments (Lezyne, 2026). The product's website highlights its wide-angle 270° visibility range and its high waterproof rating, ensuring that it remains functional while submerged for up to thirty minutes (Turner, 2025). This light is an upper-end product in the bicycle rear light market, prioritising brightness and robust construction to highlight a cyclist's presence on the road.

The Strip Drive Pro 400+ features a co-moulded, impact-resistant, rubber body with a self-forming back and ladder strap that can easily adapt to a range of seatposts (Lezyne, 2015). The system offers seven distinct output modes, ranging from a dim, energy-saving light for everyday use to an extremely bright, rapidly flashing light designed to grab attention (BikeRadar, n.d.). It is powered by a 1900 milliampere-hour (mAh) battery and utilises USB-C charging (Turner, 2025). The system operates with a single button, allowing the cyclist to switch between modes, and its memory function remembers the last used setting (BikeRadar, n.d.).

The system's main advantages include its exceptional brightness and battery efficiency. The 400-lumen light mode provides a massive boost in illumination and visibility during gloomy or foggy commutes, with an 18-hour runtime in that specific mode (BikeRadar, n.d.). For longer rides, the battery life is highly impressive, ranging from 80 hours in the 5-lumen mode to over 26 hours in the 15-lumen mode. Additionally, the wide visibility and waterproof USB-C port make it a more durable and practical choice for everyday commuting (Turner, 2025).

However, the system has several disadvantages. Physically, the supplied ladder strap is quite short, making it difficult to mount securely on particularly deep aero seat posts. Moreover, the light does not account for the seatpost angle, meaning it can point downwards and fail to project its light beam directly into the eyeline of other road users behind (BikeRadar, n.d.). More importantly, it lacks automated sensors, meaning the cyclist must manually select modes rather than the light responding automatically to riding conditions (Turner, 2025). Consequently, it cannot distinguish between a moving and an actively braking cyclist, relying completely on pre-programmed flashing patterns to project their presence rather than their actions.

The fundamental gap in this product is that it is primarily a visibility tool, not an intelligent communication and signalling device. It lacks a mechanical switch or sensor-based brake light mechanism, meaning it will continue flashing at a steady rate regardless of whether the cyclist is moving, braking, or stationary. The SBBL system addresses this gap by using direct mechanical

brake switches for immediate activation, clearly signalling to following road users when the cyclist is braking. It is worth noting that an 'Ai Alert' version of this light with brake-sensing capabilities does exist, highlighting that this specific model intentionally excluded the intelligent braking features (Turner, 2025).

2.2.2 Review of The Magene L508 Radar Tail Light



Figure 2.2 The Magene L508 Radar Tail Light (Magene, n.d.)

The Magene L508 is a smart tail light that combines rear vehicle detection with an integrated rear light, and is a budget-friendly alternative when compared to premium options like the Garmin Varia. It offers a 140-meter detection range, a horizontal 40° radar beam, and a 220° rear light

visibility angle (ProCylingUK, 2026). It uses millimetre-wave pulse signals to detect approaching vehicles and can provide visual and audible alerts through compatible bike computers, watches, or even its Utility app, making it a robust safety device rather than a simple illumination gadget (Magene, n.d.).

The L508 was developed with a slim vertical body and features a practical, impact-resistant casing with a high waterproof rating, allowing it to handle road spray and heavy rain (ProCylingUK, 2026). Moreover, it offers seven light modes, including solid, flashing, pulse, peloton, quick flash, rotation, and radar-only, with adjustable brightness and flash intervals available on the Magene app. The smart brake light function uses a built-in acceleration sensor to detect braking and automatically enters a three-second highlight mode, while ignoring vibrations, bumps, and potholes to prevent false positives (Magene, n.d.). The device also features auto-wake and auto-sleep functionality, turning on when movement occurs and conserving energy when stationary. Additionally, it supports both ANT+ and Bluetooth connections with numerous bike computers and watches (Panda Podium, n.d.).

The Magene L508's radar safety and affordability are its notable features. It provides early alerts of approaching vehicles up to 140 meters away, giving cyclists crucial time to react, and its 40° beam angle helps detect vehicles even in corners and bends (Magene, n.d.). Its smart brake sensing adds an extra layer of visibility, automatically signalling nearby road users when the cyclist slows down, which is particularly useful in group rides and commutes at constantly varying speeds (ProCylingUK, 2026). Additionally, it is a lightweight device with a USB-C charging port and provides adequate battery life ranging from 10 to 19 hours depending on the lighting mode (Panda Podium, n.d.).

On the other hand, the Magene L508 exhibits several disadvantages. For instance, its radar tracking is not as refined as that of premium competitors like Garmin, and in complex traffic scenarios such as multi-lane roads and congestion, it can occasionally produce inconsistent alerts and fail to detect other road users. Its rear light output is moderate at around 20 lumens in most modes, meaning additional lighting is required for cyclists who seek maximum visibility and illumination on the road (ProCylingUK, 2026). Most critically, the smart brake sensing is accelerometer-based, meaning it assumes braking from deceleration rather than detecting the brakes directly: this results in delayed reactions or false activations (Magene, n.d.).

The main gap in the Magene L508 is that its brake detection is deductive rather than direct. It relies on an accelerometer to predict when the cyclist is braking, which makes it unreliable since it cannot distinguish an actual braking event from another form of deceleration, meaning it may fail to register gradual stops or gentle braking (Magene, n.d.). Furthermore, it lacks a dedicated stationary hold; once the three-second highlight mode ends, the light reverts to its previous mode regardless of whether the bicycle is still stationary. The SBBL system addresses these gaps by using direct mechanical brake switches that activate instantly to braking activity, eliminating the uncertainty of an accelerometer's assumption. Additionally, the use of a state-machine framework to maintain illumination after stopping and provide the rider with manual override control ensures reliability and articulate communication that accelerometer-based systems cannot provide.

2.2.3 Review of the Sigma Sport Brake Light



Figure 2.3 The Sigma Sport Brake Light (Stead, 2015)

The Sigma Sport Brake Light is a mechanical brake-activated tail light designed to provide a direct visual indication of a cyclist's braking activity to other road users (Sigma, 2015). It is a compact, aerodynamic product that measures approximately 3 by 3 cm and weighs only 7 grams, making it one of the lightest brake light solutions on the market. The device is water-resistant and is powered

by a replaceable CR 1025 button cell battery, marketed as a simple, low-maintenance safety device for cyclists using mechanical brakes with cables (JVBike, n.d.).

The Sigma Brake Light uses direct mechanical detection; it is clamped onto the exposed inner rear brake cable with a little pre-tension applied so that the light illuminates with the slightest pull of the brake levers (JVBike, n.d.). When the brake callipers close and the cable moves, the mechanism inside completes a circuit, activating a high-powered LED that emits a bright light for the duration of the braking action (Sigma, 2015). The device requires roughly 25mm of bare brake cable to fit and is compatible with rim brakes, cantilever brakes, road callipers, and mechanical disc brakes; however, it isn't compatible with hydraulic brakes (Stead, 2015). Moreover, it has a battery life of roughly 200 hours of continuous light or around 100,000 braking processes (Sigma, 2015).

One major advantage of the Sigma Brake Light is its immediate activation, which results from its direct mechanical connection to the inner rear brake cable. The light illuminates the moment the brake cables move, providing an accurate and instant signal of the cyclist's braking activity (Sigma, 2015). Moreover, the device is extremely lightweight, easy to install with an Allen key, and has a long battery life, making it a practical solution for everyday cyclists (JVBike, n.d.).

In contrast, a major downside of the Sigma Brake Light is that it only monitors the rear brake cable, meaning that if a cyclist brakes using only the front brakes, the light will not activate. The light output is extremely low; it is incomparable to a standard rear light, and it is only noticeable if the observer is looking directly at it. Furthermore, the device is very fragile, as one reviewer reported that it "exploded into pieces" on the first firm squeeze of the brake due to over-tightening (Stead, 2015).

The Sigma Brake Light is a double-edged sword: it demonstrates the reliability of direct mechanical detection, but at the cost of functionality and visibility. It is a device that solely monitors the rear brake input, lacks stationary illumination, offers no adaptive light according to surrounding lighting, and provides unnoticeable illumination in real-world conditions (Stead, 2015). The SBBL system addresses these gaps by using dual mechanical brake switches that detect braking activity from any of the levers. It also introduces a state-machine framework that maintains illumination of the light once the bicycle is stationary, provides the cyclist with manual override control, and uses an LDR to automatically adjust the brightness according to the ambient light. By combining the reliability and accuracy of direct mechanical detection with smart state management

and adjustable brightness, the SBBL system can provide a more efficient and effective brake-light solution compared to the Sigma Brake Light.

2.3 System Requirements

2.3.1 Functional Requirements

- **Brake Detection:** The system must detect when either the left or right brake lever is pressed.
- **Brake Light Activation:** The system must immediately illuminate all three rear LEDs when braking activity is detected.
- **Brake Light Deactivation:** The system must switch off all the LEDs when both brake levers are released, provided the bicycle is not stationary after a braking incident.
- **Motion State Detection:** The system must distinguish between a moving and a stationary bicycle.
- **Stationary Illumination:** The system must keep the LEDs illuminated after the bicycle stops following a braking event, even if the brake levers are released.
- **Indefinite Stationary Illumination Hold:** The stationary illumination must remain active without an automatic cut-off time until the cyclist manually switches it off or starts cycling again.
- **Manual OFF override:** The system must allow the cyclist to manually deactivate the stationary illumination while stationary. Moreover, the manual OFF override must remain active while the bicycle is stationary, preventing further brake actions from reactivating the LEDs until movement resumes.
- **Movement Reset:** The system must reset the stationary illumination hold and manual OFF override when the bicycle starts moving again, returning to the normal brake light operation.
- **No Illumination during Normal Movement:** The LEDs must remain off during normal movement when the brakes are not applied; the system must not function as a continuously illuminating rear light.
- **Ambient-light Brightness Adjustment:** The system must measure the environmental light level using a light-dependent resistor (LDR), then adjust the brightness of the LEDs accordingly using pulse-width modulation.

- **Brake Priority over Ambient Light:** The measurement of the ambient light must only determine LED brightness when illumination is required. It must not independently activate the LEDs or override the braking and stationary illumination.
- **LDR Fault Detection:** The system must identify a potential fault when the light sensor produces persistent abnormal readings.
- **Safe Light Brightness Fallback:** When a potential LDR fault is confirmed, the system must default to a fixed LED brightness only when illumination is required.
- **Fault Recovery:** The system must automatically clear the fault flag and restore normal ambient brightness control when the sensor reading is back to normal.
- **Power Management:** The system must enter sleep mode with low power usage when the brake light is not required, and the bicycle is not in an active illumination state.
- **Interruption-based Wake-Up:** The system must wake from sleep when the brake is applied or when the motion state changes.
- **Wake-Up Illumination Control:** Waking the system mustn't automatically illuminate the LEDs; illumination must only occur if braking is detected or a valid stationary condition exists.

2.3.2 Non-Functional Requirements

- **Responsiveness:** The system must activate the brake lights immediately when a brake lever is pressed, with no noticeable delay.
- **Reliability:** The system must behave consistently during braking activity and motion state changes without requiring a restart.
- **Usability:** The system must operate automatically during normal cycling, requiring no manual input except for the optional manual OFF override.
- **Power Efficiency:** The system must reduce unnecessary power usage by entering sleep mode when illumination isn't required.
- **Safety Prioritisation:** Braking and stationary illumination must always take priority over ambient-light adjustment, and a sensor fault must never prevent the brake light from illuminating.

- **Non-Blocking Operation:** The system must process inputs and manage its lighting functions without relying on blocking delay functions that would prevent it from responding promptly to braking or motion changes.
- **Straightforward and Simple Operation:** The system must require only a single manual override from the rider, explicitly leaving the rest of the functions automatic for the rider's convenience and safety.
- **Fault Tolerance:** The system must detect at least one identifiable sensor fault and respond with a safe fallback rather than relying on potentially unreliable sensor information.
- **Testability:** Each functional specification, including brake detection, motion recognition, ambient brightness adaptability, manual override, fault detection, and power management must be independently testable in the simulation environment.
- **Hardware Requirements:** The system must operate in the Tinkercad simulation environment using an Arduino Uno, mechanical pushbutton switches, a slide switch, three LEDs with current-limiting resistors, and an LDR with a voltage-divider resistor.

2.4 System Architecture

The SBBL system follows a three-stage architecture consisting of inputs, processing, and outputs. The structure separates the physical signals that the system receives from the decisions made by the microcontroller and the resulting LED output, which keeps the design organised and makes each stage easier to understand, develop, test, and modify. The three stages continuously work together: the inputs provide information about the bicycle's state, the Arduino processes that information to determine which operation mode is appropriate, and the outputs reflect that decision through the LEDs.

The input stage consists of four components that provide data about the bicycle and cyclist's actions. Two mechanical pushbutton switches represent the left and right brake levers; when either is pressed, the system interprets this as a braking event. The slide switch represents the bicycle's motion state, distinguishing between moving and stationary conditions. A light-dependent resistor (LDR) is used to measure the surrounding light level, allowing the system to determine whether it is daytime, dusk, or nighttime. Finally, the manual OFF pushbutton gives the cyclist direct control over the stationary illumination, allowing them to turn it off when it is no longer needed. These

inputs are read by the Arduino, and none of them directly controls the LEDs on its own; instead, they provide the information needed by the Arduino to make decisions.

The Arduino Uno is the system's processor and central controller. It continuously reads the values of all four inputs, determines the current operating mode of the system, selects the suitable LED brightness based on the ambient-light reading, and controls the outputs accordingly. Moreover, it also manages the system's advanced functions, including continuously illuminating the LEDs after the bicycle stops, resetting the stationary hold when movement resumes, handling the manual OFF override, and entering sleep mode when illumination is not needed. The processing stage is explicitly designed to remain responsive constantly; it uses non-blocking logic so that it can continue monitoring the inputs and respond to changes without delay. Finally, this stage determines how the system behaves, meaning changes to the system are made by modifying the program, not the circuit structure.

The output stage consists of three red LEDs, which altogether represent the bicycle's rear brake light. These LEDs are not simply switched on or off; they are controlled using pulse-width modulation (PWM), which allows the Arduino to vary the brightness in response to the ambient-light reading. When braking activity is detected, the Arduino illuminates all three LEDs instantly at the appropriate brightness. When the bicycle stops after a braking event, the LEDs remain on until the cyclist either presses the manual OFF button or resumes cycling. When neither braking nor stationary illumination is required, all the LEDs are switched off. By managing all three LEDs together, the system presents a single, consistent braking signal rather than a fragmented output, which makes it easier for other road users to interpret the signals.

The system's architecture is modular: each input is read independently, the processing logic is centred in one microcontroller, and each output is controlled through a single, coordinated instruction. The modular structure allows each functional section of the system, including brake detection, motion state recognition, ambient-light adaptation, manual override, and LED output, to be developed, tested and verified independently before integration, reducing the risk of introducing errors when the modules are combined.

2.4.1 Block Diagram

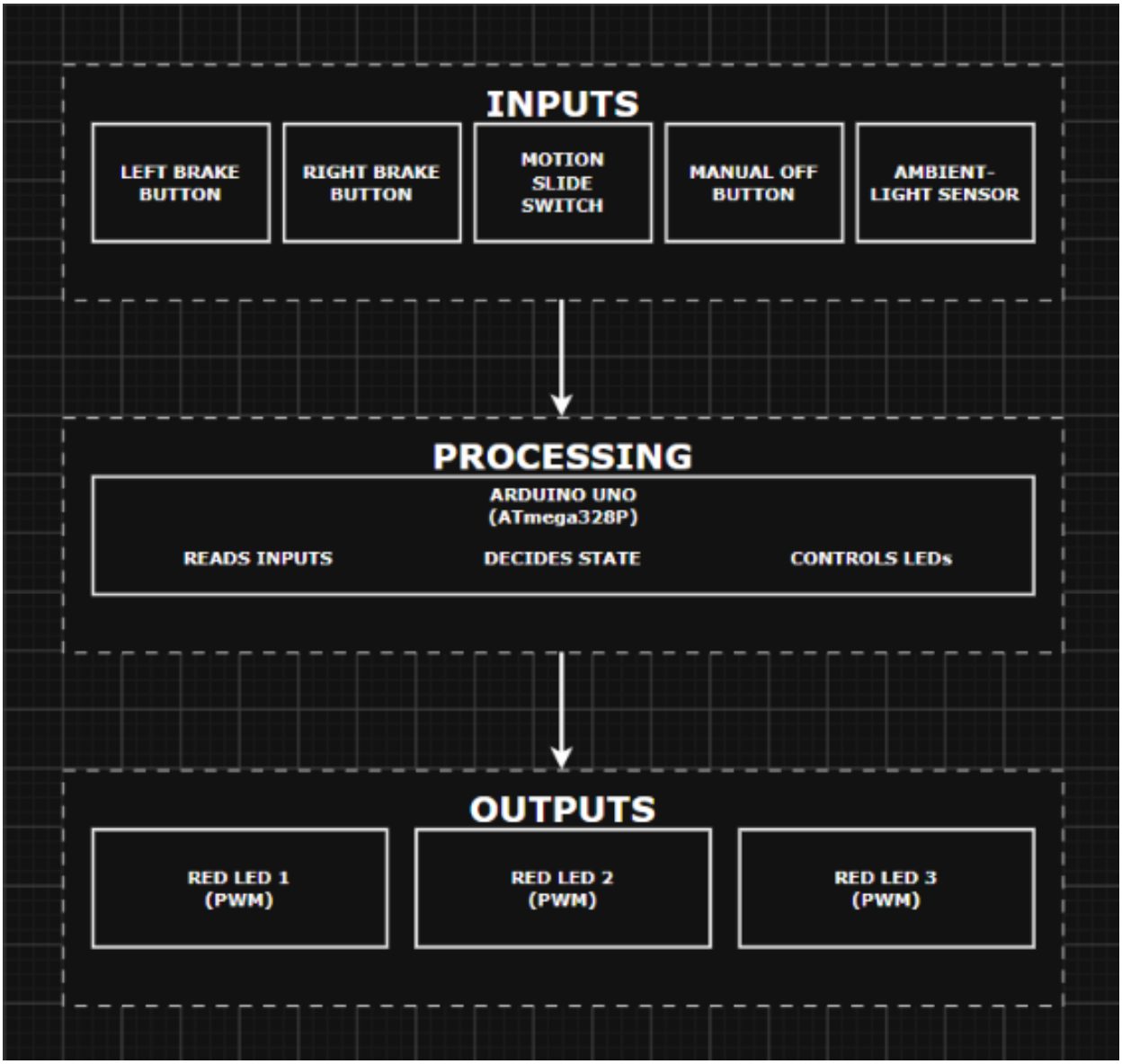


Figure 2.4 SBBL system architecture block diagram showing the input, processing, and output stages.

2.4.2 State-Machine Logic

The SBBL system is designed around a state machine, which is a design approach where the system is always in exactly one state at a time. Transitions between these states are triggered by specific combinations of input conditions. This approach ensures that the system's behaviour is consistent, predictable, and easy to debug, as the system cannot be in two conflicting modes at once. To ensure the system remains responsive, the state machine uses the millis() function to

manage timing instead of relying on the delay() function. This prevents the Arduino from lagging and ensures it continuously monitors all inputs for braking, motion, or manual overrides with no interruption.

The system operates across four distinct states:

- **CYCLING:** The bicycle is moving, and the brakes are released. The LEDs remain OFF, since illumination is not required.
- **BRAKING:** A brake lever is pressed, either while moving or stationary. The LEDs turn ON immediately, with their brightness determined by the ambient-light reading.
- **STATIONARY:** The bicycle has transitioned from moving to stationary following a braking event. The LEDs remain ON indefinitely to warn other road users, even if the cyclist releases the brakes.
- **DEACTIVATED:** The cyclist has pressed the manual OFF button while stationary. The LEDs switch OFF and remain OFF, ignoring further brake activity, until the bicycle starts moving again.

2.4.3 State Transition Diagram

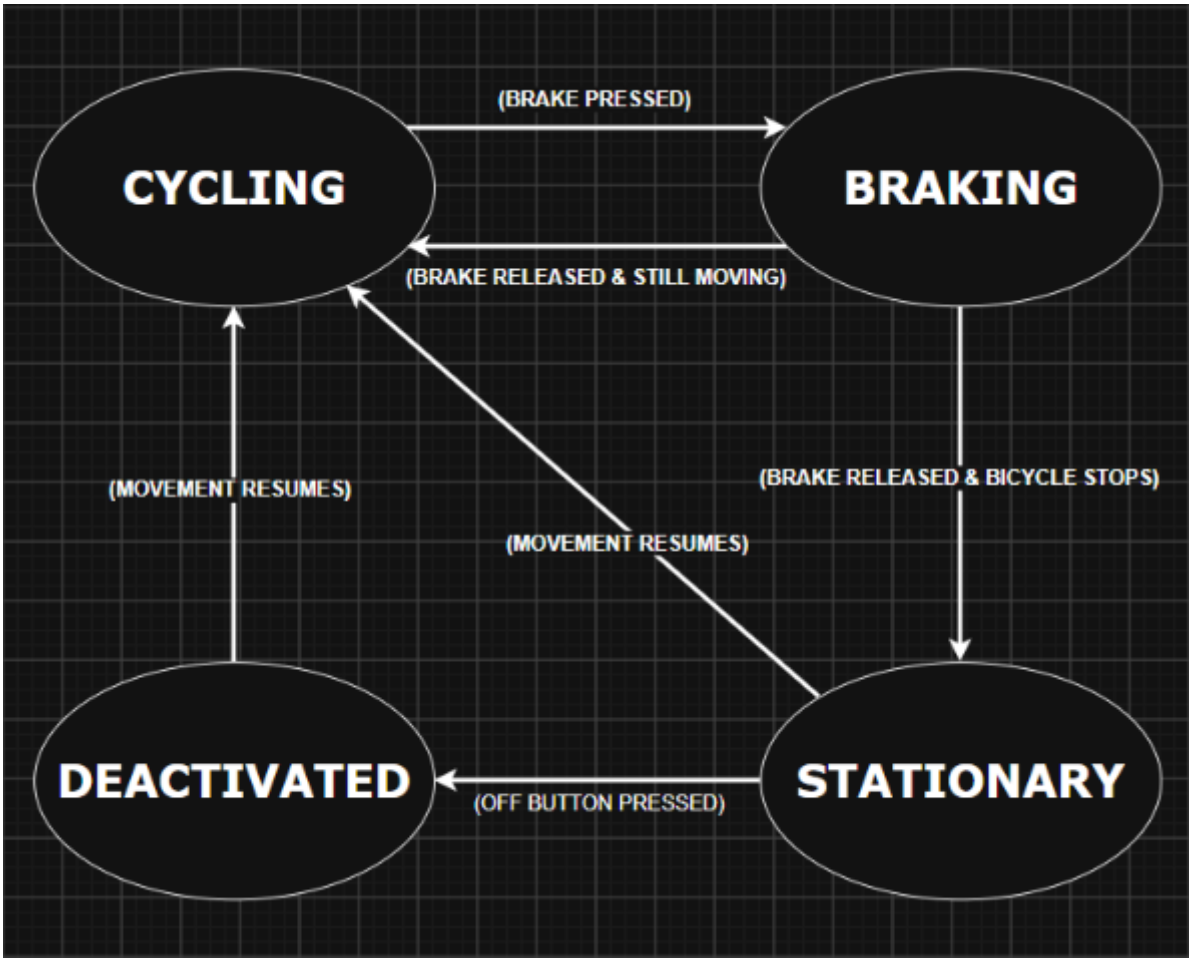


Figure 2.5 State transition diagram showing the four operating modes (CYCLING, BRAKING, STATIONARY, DEACTIVATED) and the triggers that cause the transitions between them.

2.4.4 State Transition Table

Table 2.1 State transition rules for the SBBL state machine.

CURRENT STATE	TRIGGER	NEXT STATE
CYCLING	A brake button is pressed.	BRAKING
BRAKING	Brakes are released, and the bicycle is still moving.	CYCLING

BRAKING	Brakes are released, and the bicycle transitions from moving to stationary after a braking event.	STATIONARY
STATIONARY	OFF button is pressed while stationary.	DEACTIVATED
STATIONARY	Bicycle starts moving	CYCLING
DEACTIVATED	Bicycle starts moving	CYCLING

Having established the research gaps, system requirements, and architectural design of the SBBL system in this chapter, the next chapter will discuss the methodology and development approach used to implement this design.

Chapter 3: Methodology and Simulation Implementation

3.1 Introduction

This chapter details the methodology, development environment, and simulation implementation of the Smart Bicycle Brake Light (SBBL) system. It begins with an outline of the five-phase development approach used for the project, followed by an overview of the Tinkercad simulation environment and the component inventory. Furthermore, this chapter comprehensively breaks down each implementation phase, explaining the circuit construction and connections, code logic, and real-world comparisons. Finally, it summarises the fully integrated system and the technical challenges faced during the developmental process.

3.2 Development Methodology

The development of the SBBL system utilised an incremental, modular phased approach. The system was divided into five distinct phases: each phase introduced a specific functionality, and after each phase, the circuit was developed and tested in isolation; it was integrated into the next phase. This methodology produced an intelligent brake-light system through progressive development, ensuring all the phases were fully functional and integrated to produce the final system.

3.3 Simulation Environment and Component Inventory

3.3.1 Tinkercad Circuits

The project was developed and evaluated using Tinkercad Circuits. It is a free, web-based simulation platform that simplifies the construction of electronic circuits and supports C++ programming for the Arduino Uno. It is highly accessible and well-suited for logic testing and verifying accurate component interactions; however, it is important to note that Tinkercad also has its limitations. For instance, it doesn't accurately simulate how the ATmega328P microcontroller saves power in a low-power sleep state, nor does it simulate environmental conditions in the real world.

3.3.2 Component Inventory

The system uses a specific set of components to replicate the real-world behaviour of a smart bicycle brake light.

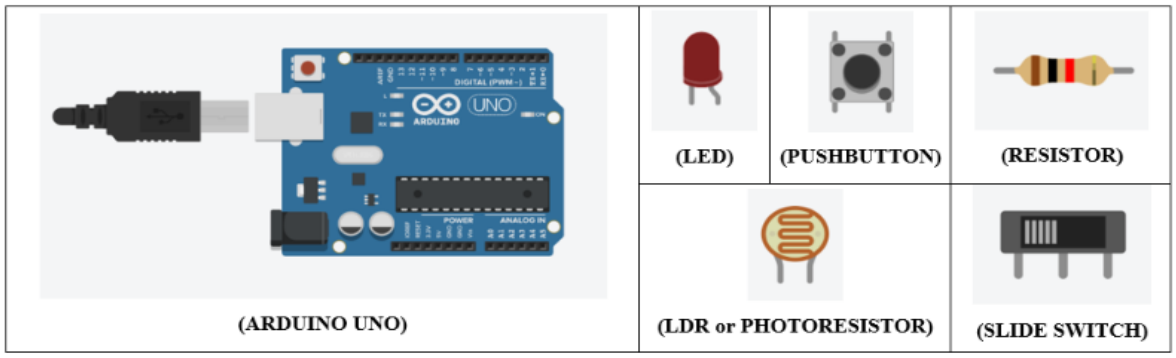


Figure 3.1 SBBL Circuit Components

Table 3.1 Component inventory for the SBBL simulation, detailing their real-world equivalents and roles in the system.

COMPONENT	QUANTITY	REAL-WORLD APPLICATION	ROLE IN THE SYSTEM
Arduino Uno	1	The central processing unit (CPU) or microcontroller.	This reads all the sensor inputs, executes the state machine logic, and controls the LED outputs.
Pushbuttons	3	Two represent the brake levers, and the other represents the Manual OFF switch.	The first two simulate the left and right brake levers. The third one allows the cyclist to manually deactivate the stationary light.
Slide Switch	1	A motion detector	This simulates whether the bicycle is moving (HIGH) or stationary (LOW).
LDR (Photoresistor)	1	Ambient-light sensor	This measures the surrounding light levels (daylight, dusk, nighttime) to adjust the LED brightness.
Red LEDs	3	The rear brake light	This illuminates to signal braking activity and a stationary state.
Resistors	4	The first 3 are current-limiting resistors for the LEDs. The last one is a voltage-divider resistor for the LDR.	The first three resistors protect the LEDs from drawing too much current. The last one works with the LDR to create a

			measurable analog voltage for the Arduino.
Wires	Unlimited	Electrical wiring	This connects the components to the Arduino board.

3.4 Phase Evaluations

3.4.1 Phase 1 Implementation: Core Brake Detection

The primary aim of this phase was to establish the system's core trigger: detecting when a cyclist applies the brakes and instantly illuminating the rear LEDs in response.

For this phase, an Arduino Uno was connected to two mechanical pushbuttons: real bicycles have separate front and rear brake controls, so using two buttons ensured that the system could demonstrate that it responds correctly to either brake lever. Both pushbuttons were wired to Digital Pin 2 (D2) using the Arduino's internal pull-up resistor (INPUT_PULLUP). Three red LEDs were connected to Pins 9, 10, and 11 (D9, D10, D11), each paired with a current-limiting resistor. Notably, in the simulation, the green wires represent the connections for this phase.

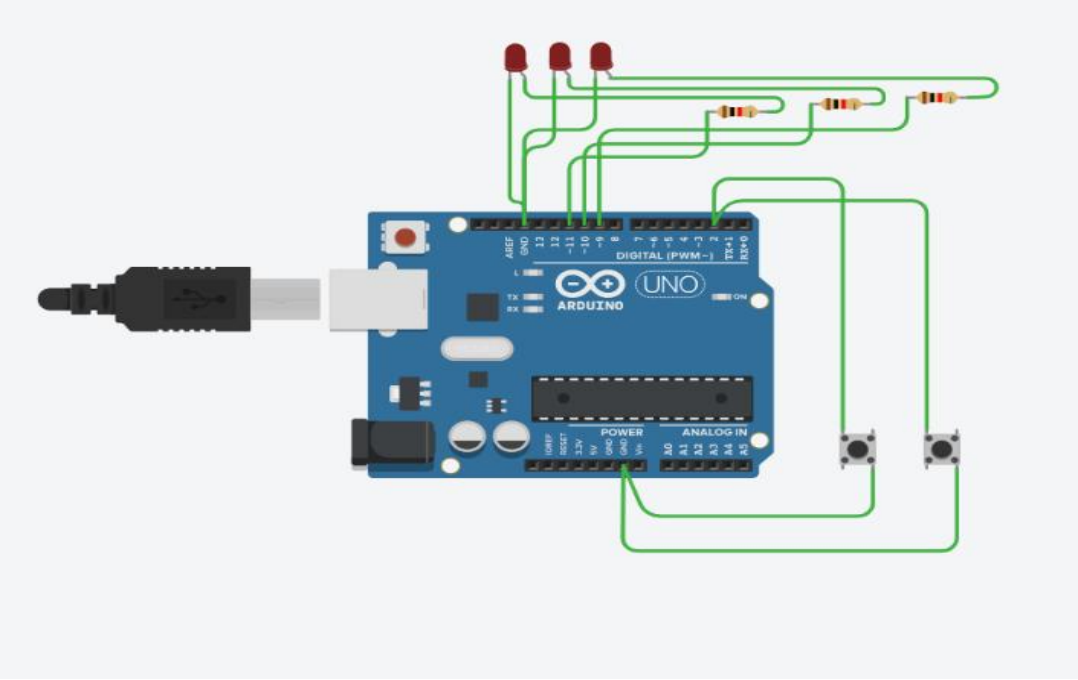


Figure 3.3 Phase 1 circuit showing the dual brake buttons and the three red LEDs

The code for Phase 1 establishes the foundation for the CYCLING to BRAKING state transition. The setup() function configures D2 as an input with a pull-up resistor, and D9, D10, and D11 as outputs. In the loop, the Arduino continuously reads the state of D2: because of the INPUT_PULLUP configuration, the pin remains HIGH when the brakes are released, and drops to LOW when either brake is pressed. When the LOW signal is detected, the Arduino executes an if statement that sends a HIGH signal to all three LEDs, turning them on. When the buttons are released, the pin returns to HIGH, switching off the LEDs.

```
const int brakePin = 2;

const int ledPin1 = 9;
const int ledPin2 = 10;
const int ledPin3 = 11;

void setup()
{
  // Initialising braking as input
  pinMode(brakePin, INPUT_PULLUP);
  // Note: 'INPUT_PULLUP' enables the Arduino's internal pull-up resistor

  // Intialising the LEDs as output
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(ledPin3, OUTPUT);
}

void loop()
{
  // Reading the brake input
  int brakeState = digitalRead(brakePin);
  // Note: Since we used 'INPUT_PULLUP:
  // 'LOW' means a pressed button, or in our case braking
  // 'HIGH' means both buttons are released, so no braking

  // Conditional statements detect brake use and its response
  if (brakeState == LOW){
    digitalWrite(ledPin1, HIGH);
    digitalWrite(ledPin2, HIGH);
    digitalWrite(ledPin3, HIGH);
  }
  else {
    digitalWrite(ledPin1, LOW);
    digitalWrite(ledPin2, LOW);
    digitalWrite(ledPin3, LOW);
  }
}
```

Figure 3.5 Code Excerpt: The foundational code used to read the brake inputs and illuminate the LEDs.

Figure 3.6 Code Excerpt: The foundational code used to read the brake inputs and illuminate the LEDs.

In a real-world scenario, these two pushbuttons perfectly simulate the electrical switches that are often integrated with bicycle brake levers. When a cyclist pulls the brake lever, it closes a switch, sending a signal to the microcontroller, resulting in the illumination of the brake light LEDs. This phase directly mimics the most important feature of this project: the immediate illumination of the brake light the moment the brake lever is engaged, warning following road users that the bicycle is slowing down.

3.4.2 Phase 2 Implementation: Motion Recognition and Manual Override

The main aim of Phase 2 was to introduce intelligent behaviour by adding motion recognition and an override control. While Phase 1 only reacted to the brake levers, Phase 2 aimed to make the system aware of whether the bicycle was moving or stationary. This allowed the system to maintain illumination after a stop and introduced a manual override button so that the cyclist could turn off the stationary light when finished with their ride.

Building on Phase 1's circuit, a slide switch was added to Digital Pin 3 (D3) to simulate the bicycle's changes in motion. Unlike the buttons, this switch was configured as a standard INPUT because it connects directly to 5V and GND (the 'ground with 0 volts'), providing a definite HIGH (Moving) or LOW (Stationary) signal. Additionally, a manual OFF pushbutton was added to Digital Pin 4 (D4) using the Arduino's internal pull-up resistor. Notably, in the simulation, the red wires represent the new connections added in this phase.

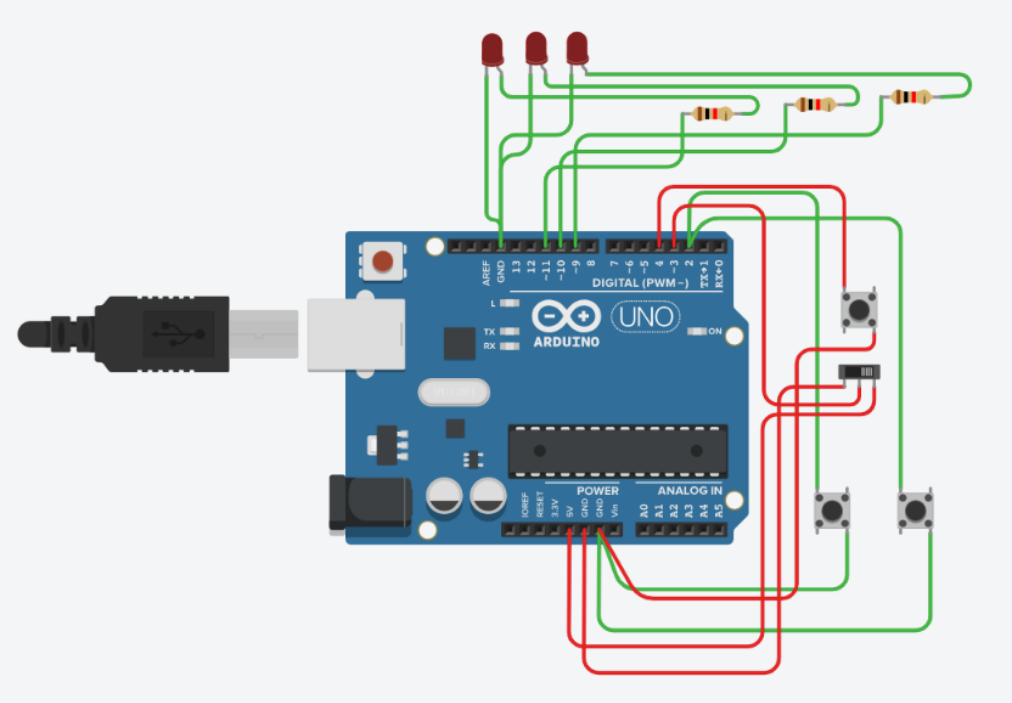


Figure 3.7 Phase 2 circuit adding the motion slide switch and manual override button.

This phase significantly upgrades the state-machine logic by introducing memory variables because the system now needs to remember if a braking event occurred before the bicycle stopped. Moreover, several boolean variables were also introduced.

The code detects the exact moment the bicycle stops (justStopped) and the exact moment it starts moving again (justStartedMoving). If the bicycle stops after a braking event, the holdActive variable is set to true, keeping the LEDs on indefinitely. However, the most important piece of state machine logic is the final condition that decides if the LEDs should be on:

“bool lightOn = !manualOff && (braking || (stopped && holdActive));”

This single line dictates that if the following conditions are met, then the LEDs will turn on: if the cyclist is braking, or if the bicycle is stationary and the holdActive variable is true, provided that the manualOff variable is false. Crucially, the !manualOff condition at the beginning of the line ensures that if the cyclist presses the OFF button, the LEDs are forced off, regardless of any other condition, until the bicycle starts to move again (which resets the entire system).

```
// Resetting the system when movement resumes
// When the bicycle transitions from stationary (LOW) to moving (HIGH), the Arduino resets the stationary lighting system
if (justStartedMoving){
  holdActive = false;
  // Cancels the stationary light display
  manualOff = false;
  // Clears the manual OFF override so the system is ready for the next stop
  brakingEventSeen = false;
  // Clears the previous braking event so that a new one can be detected during the continuation of the ride
}

// Recording a braking event
if (braking){
  brakingEventSeen = true;
}

// Starting the stationary hold after braking
// It is started when the simulated 'bicycle' becomes stationary
if (justStopped && brakingEventSeen){
  holdActive = true;
}

// Overiding the manual OFF button
// When the bike is stationary (stopped) and the OFF button is pressed, the Arduino activates the manual OFF override
if (stopped && offButtonJustPressed){
  manualOff = true;
}

// This section of code combines the braking and movement information above to determines whether the LEDs should be ON or OFF
bool lightOn = !manualOff && (braking || (stopped && holdActive));
digitalWrite(ledPin1, lightOn ? HIGH : LOW);
digitalWrite(ledPin2, lightOn ? HIGH : LOW);
digitalWrite(ledPin3, lightOn ? HIGH : LOW);
// The 'lightOn ? HIGH : LOW' is a shorter way of saying if 'lightOn' is true use 'HIGH' and if it is false use 'LOW'
// The '?' is what starts the conditional expression

previousMotionState = motionState;
// Note: This updates the previous motion state as the current motion state for the next loop
```

Figure 3.9 Code Excerpt: The logic added to handle the stationary illumination hold and the manual override without blocking the loop.

Figure 3.10 Code Excerpt: The logic added to handle the stationary illumination hold and the manual override without blocking the loop.

In a real-world scenario, the slide switch simulates a motion detector (like the ones found in modern e-bikes); it mimics what happens when a cyclist stops at a junction and releases the brake levers. The holdActive state ensures that the brake lights remain illuminated, warning approaching vehicles that the bicycle is stationary. Furthermore, the Manual OFF button simulates a physical kill switch or a battery-saving mode that a rider would use when they are no longer cycling.

3.4.3 Phase 3 Implementation: Intelligent Lighting Management

The main aim of Phase 3 was to make the brake light adaptive to its environment. While Phases 1 and 2 ensured the light turned on at the right time, Phase 3 focused on the LEDs' brightness. By introducing an ambient-light sensor, the system aimed to automatically adjust the LED brightness based on whether it was daytime, dusk, or nighttime, ensuring optimal visibility.

Building on Phase 2's circuit, an LDR and a voltage-divider resistor were added. The LDR was connected to Analog Pin A0, creating a voltage divider that allows the Arduino to read the surrounding light levels. To enable variable brightness, the three LEDs on Pins 9, 10, and 11 were switched from standard digital outputs to PWM (Pulse Width Modulation) outputs. Notably, in the simulation, yellow wires represent the new connections added in this phase.

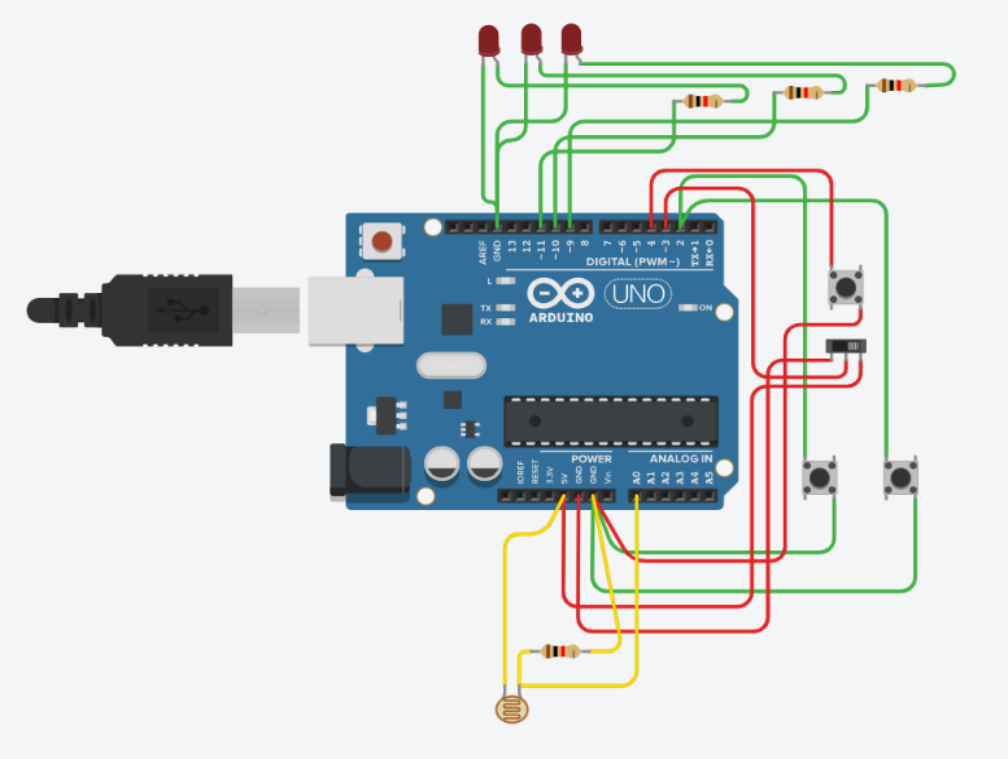


Figure 3.11 Phase 3 circuit introducing the LDR for ambient-light sensing.

This phase upgraded the output logic by replacing digitalWrite(), which only switched the LEDs ON or OFF, with analogWrite(), which outputs different brightness values from 0 to 255. Moreover, the code uses analogRead(A0) to determine the ambient light level and classifies it into three distinct environmental states using manually determined thresholds:

- **Daylight:** Input reading ≥ 620 (Output PWM Brightness = 180)
- **Dusk:** Input reading between 271 and 619 (Output PWM Brightness = 218)
- **Nighttime:** Input reading ≤ 270 (Output PWM Brightness = 255)

The state-machine logic that determines when the lights turn on (bool lightOn = !manualOff && (braking || (stopped && holdActive));) remains unchanged. The LDR only determines how

brightly the LEDs shine when they are already required. The final output is controlled by passing the selected ledBrightness value into the analogWrite() function for each LED.

```
// Identifying the surrounding light conditions
int ledBrightness;

// Setting the LED brightness based off the boundaries for daylight and nighttime
// If the reading is 620 or above, the Arduino selects daylight brightness
if (lightLevel >= daylightThreshold){
  ledBrightness = daylightBrightness;
}
//  If the reading is 270 or below, the Arduino selects nighttime brightness
else if (lightLevel <= nightThreshold){
  ledBrightness = nightBrightness;
}
// If neither condition is met, the Arduino selects dusk brightness
// Since it is the remaining values between the nighttime highest boundary and daylight lowest boundary
else {
  ledBrightness = duskBrightness;
}
```

Figure 3.13 Code Excerpt: The code added to classify the LDR reading into daylight, dusk, or nighttime brightness levels.

```
// Determining whether the brake lights should be ON or OFF
bool lightOn = !manualOff && (braking || (stopped && holdActive));
// The LEDs illuminate when braking or when the stationary hold is active
// However, the manual OFF override takes priority and keeps the LEDs switched OFF

// Controlling the LED brightness
if (lightOn){
  analogWrite(ledPin1, ledBrightness);
  analogWrite(ledPin2, ledBrightness);
  analogWrite(ledPin3, ledBrightness);
}
// If the light is on, then the LEDs will use the brightness selected according to the LDR reading from A0

else{
  analogWrite(ledPin1, 0);
  analogWrite(ledPin2, 0);
  analogWrite(ledPin3, 0);
}
// If the brake lights should be OFF, all three LEDs receive a brightness value of zero
/* In this section, 'analogWrite()' is used instead of 'digitalWrite()'.
'analogWrite()' uses PWM (Pulse Width Modulation) to control the
LED brightness using values between 0 and 255.
The selected brightness depends on the surrounding light detected by the LDR.

'DigitalWrite()', on the other hand, simply switches the LEDs ON or OFF
using 'HIGH' and 'LOW' respectively */
```

Figure 3.15 Code Excerpt: The code used to apply the calculated brightness to the LEDs using analogWrite() only when the lightOn state machine logic is true.

In a real-world scenario, the LDR simulates the ambient-light sensors found in high-end bicycle lights. During the day, the brake light only needs moderate illumination to be seen against the sun. However, at night, a maximum brightness that illuminates the cyclist's braking actions whilst not glaring at the other road users is necessary. This phase mimics a smart brake light that automatically increases its intensity in dark conditions, ensuring the cyclist remains highly visible while maintaining safety for following traffic.

3.4.4 Phase 4 Implementation: Power Management and Sleep Mode

The primary aim of Phase 4 was to introduce a power management system to conserve energy and save battery life. While the previous phases addressed fundamental functional requirements and

ensured the system reacted effectively and efficiently, they left the Arduino constantly awake, draining the battery unnecessarily. This phase implemented a low-power sleep mechanism that allowed the microcontroller to conserve energy when the brake light was not required, while ensuring it could wake up immediately when the cyclist applied the brakes or started moving again.

No new components were added in this phase, as the existing circuit from Phase 3 already contained all the essential inputs. However, the connection strategy shifted to use the Arduino's hardware interrupt capabilities: specifically configuring the brake buttons on D2 and the motion slide on D3 to act as wake-up triggers.

To implement this, two AVR libraries were included: <avr/sleep.h> and <avr/interrupt.h>. The code introduces two volatile boolean variables (brakeWakeEvent and motionWakeEvent) that act as triggers. The attachInterrupt() function is used in the setup() block to link D2 (using the FALLING edge) and D3 (using the CHANGE edge) to two Interrupt Service Routines (ISRs). These ISRs are kept very short because they have to immediately react and respond to the wake-up triggers. They only set up the specific wake-event trigger to true, leaving the actual LED logic to the main loop.

A new function, enterSleepMode(bool lightIsOn), was created. This function checks if it is safe to enter sleep mode (the brakes are released and the LEDs are off). If safe, it sets the Arduino to its deepest sleep mode (SLEEP_MODE_PWR_DOWN). It then enables sleep by calling the sleep_enable() function, turns on interrupts using the sei() function, and pauses the processor (sleep_cpu()). When an event or trigger wakes it up, it turns the sleep function off (sleep_disable()).

The state-machine logic is preserved. Waking up does not automatically turn the LEDs on; it simply allows the Arduino to resume the loop() and re-evaluate the inputs. If braking activity woke up the system, the existing lightOn logic determines if the light should actually illuminate (which is especially important if the manualOff override is active).

```
  // Configuring the external interrupt for the brake button
  attachInterrupt(digitalPinToInterrupt(brakePin), wakeOnBrake, FALLING);
  // Detects when the brake button changes from HIGH to LOW

  // Configuring the external interrupt for the motion switch
  attachInterrupt(digitalPinToInterrupt(motionPin), wakeOnMotion, CHANGE);
  // Detects whenever the motion switch changes between HIGH and LOW

}

// INTERRUPT SERVICE ROUTINES (USED TO WAKE THE ARDUINO FROM SLEEP)

// Interrupt service routine for detecting a brake button press
void wakeOnBrake() {
  brakeWakeEvent = true;
  // Records that a brake related wake up event has occurred
}

// Interrupt service routine for detecting a change in the motion switch
void wakeOnMotion() {
  motionWakeEvent = true;
  // Records that a motion related wake up event has occurred
}
```

Figure 3.17 Code Excerpt: The configuration of the hardware interrupts on pins D2 and D3, with their Interrupt Service Routines (ISRs) used to wake up the Arduino when a wake-up event is triggered.

Figure 3.18 Code Excerpt: The configuration of the hardware interrupts on pins D2 and D3, with their Interrupt Service Routines (ISRs) used to wake up the Arduino when a wake-up event is triggered.

```
// FUNCTION FOR ENTERING POWER-DOWN SLEEP MODE

void enterSleepMode(bool lightIsOn)
{
  if (digitalRead(brakePin) == LOW || lightIsOn){
    return;
  }
  // This conditional statement checks if a brake is pressed or if the LEDs are on
  // If either of these conditions are true, then the 'return' exits the function immediately

  set_sleep_mode(SLEEP_MODE_PWR_DOWN);
  // Configures the microcontroller to enter power down sleep mode, reducing the power consumption significantly

  cli();
  // Disables the global interrupts temporarily, to prevent an interrupt from occurring between the final safety check and the Arduino entering sleep

  if (digitalRead(brakePin) == HIGH && !lightIsOn && !brakeWakeEvent && !motionWakeEvent){
    sleep_enable();
    // Enables the sleep function
    sei();
    // Re-enables global interrupts
    sleep_cpu();
    // Puts the Arduino into sleep mode
    sleep_disable();
    // Disables the sleep function after the Arduino wakes up
  }
  // This conditional statement performs a final safety check before sleeping.
  // If all the conditions are true, then it will proceed to enter sleep mode to save power
  /* Conditions include checking if the brake is released, the LEDs are OFF,
  no brake wake event has occurred, and no motion related event has occurred */

  else {
    sei();
  }
  // If the final safety check fails, the Arduino doesnt sleep, and the interrupts and re-enabled
}
```

Figure 3.19 Code Excerpt: The power-down sleep sequence: it highlights the safety checks and core commands used to stop and resume the microcontroller.

Figure 3.20 Code Excerpt: The power-down sleep sequence: it highlights the safety checks and core commands used to stop and resume the microcontroller.

```
  // Checking whether the Arduino can enter power down sleep mode
  enterSleepMode(lightOn);
}
```

Figure 3.21 Code Excerpt: At the end of the main loop, the function call evaluates the current system state and triggers sleep mode if the conditions are safe.

To verify the sleep and wake-up logic during the development phase, temporary testing code utilising the Serial Monitor was incorporated. This included initialising the monitor (Serial.begin(9600)) and printing messages to confirm when the system entered sleep mode or when a specific brake or motion interrupt triggered a wake-up. Crucially, while the testing code proves the logical execution, Tinkercad does not accurately simulate the physical power-down state of the microcontroller. Consequently, the simulation rapidly loops through the sleep or wake sequence rather than remaining idle. As a result, the Serial Monitor code was strictly used for verification and was removed in the final product. The results of these tests, the Serial Monitor's

outputs, and the simulation's limitations in accurately testing the power-down state will be extensively evaluated in Chapter 4.

In a real-world scenario, this phase mimics the power management systems found in modern and advanced vehicles and e-bikes. The SBBL system sleeps when the bicycle is idle. Furthermore, the interrupt-based wake-up logic simulates the instant responsiveness required for safety. By sleeping when the bicycle is inactive, but instantly waking up when brake activity or a change in motion state occurs, the system ensures optimal battery use without ever compromising the rider's safety.

3.4.5 Phase 5 Implementation: LDR Fault Detection and Safety Fallback

The main aim of Phase 5 was to integrate fault tolerance, ensuring that the system still operates even if the ambient light sensor fails. While the third phase allowed the system to adapt to environmental light, a faulty sensor could produce abnormally high readings, causing the system to incorrectly assume it is daytime, dimming the brake lights. This phase implemented a fail-safe mechanism: if the LDR produces an abnormal reading for a considerable period, the system flags a fault and forces the LEDs to their maximum brightness, ensuring that the brake light signals remain clearly visible at all times.

No new components were added in this phase, as the existing circuit from Phase 4 already contained the LDR and all the necessary connections. On the other hand, to implement fault detection, new variables were added: maximumValidLightLevel (set to 900), LDRFaultDelay (set to 2000 milliseconds), and the tracking variables LDRFault, abnormalLightTiming, and abnormalLightStart.

The code continuously reads the LDR value using the analogRead() function, and if the reading exceeds 900, the system uses the millis() function to start a non-blocking timer. If the abnormal reading persists for 2 seconds, the LDRFault variable is set to true. If the reading falls below the limit before the 2 seconds elapse, the timer and fault flag are reset.

This phase preserves the core state-machine structure: the fault detection only changes how brightly the LEDs shine, not when they turn on. When a fault is detected, the system overrides the normal daylight/dusk/nighttime brightness calculation and sets the ledBrightness to 255 (maximum). The final lightOn condition remains untouched, and the LEDs will only illuminate at

this maximum fallback brightness if the cyclist is actually braking or stationary (and the manual OFF is not active).

```
// LDR FAULT DETECTION
if (lightLevel > maximumValidLightLevel){
  // Starts timing when an abnormal reading is first detected
  if (!abnormalLightTiming){
    abnormalLightTiming = true;
    abnormalLightStart = millis();

    Serial.println("WARNING: ABNORMAL LDR READING DETECTED");
    Serial.print("LDR READING: ");
    Serial.println(lightLevel);
    Serial.println("CHECKING IF THE ABNORMAL LEVEL PERSISTS");
}
  // Confirms the potential fault after 2 seconds
  if (millis() - abnormalLightStart >= LDRFaultDelay){
    if (!LDRFault){
      LDRFault = true;
      Serial.println("LDR FAULT CONFIRMED!!!");
      Serial.println("ACTIVATING THE MAXIMUM FALLBACK BRIGHTNESS");
    }
  }
}

else {
  // Resets the timer when the reading returns to the normal range
  abnormalLightTiming = false;
  // Clears the potential fault
  if (LDRFault){
    LDRFault = false;
    Serial.println("LDR READING RETURNED TO NORMAL");
    Serial.println("NO LDR FAULTS, RESTORING THE AMBIENT BRIGHTNESS CONTROL");
  }
```

Figure 3.23 Code Excerpt: The final safety logic ensuring maximum brightness during LDR sensor failure.

To verify the fault detection and fallback logic during the development phase, specific testing methods were implemented. Because the normal maximum threshold of 900 was unreachable in the Tinkercad simulation (the highest LDR reading achievable was approximately 679), the maximumValidLightLevel was temporarily lowered to 630. This adjustment enabled manual adjustment of the Tinkercad LDR slider to its maximum brightness setting to intentionally trigger an abnormal reading. Additionally, temporary Serial.print() statements were added in the code to output real-time status messages to the Serial Monitor. This enabled observation of the 2-second confirmation timer in action, verifying that the LDRFault flag was successfully set, and confirming that the system activated the maximum fallback brightness (255). Once the testing was complete, the threshold was restored to its original value of 900, and the temporary Serial Monitor code was removed from the final system. Further details on these tests and the simulation's behaviour will be evaluated in Chapter 4.

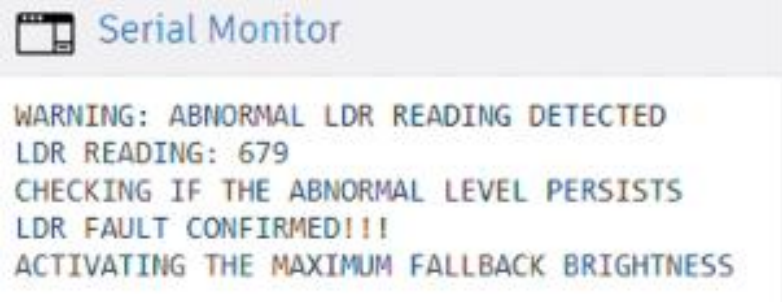


Figure 3.25 Code Excerpt: The Serial Monitor verifies that the system detects an abnormal LDR reading and triggers the fallback brightness.

Figure 3.26 Code Excerpt: The Serial Monitor verifies that the system detects an abnormal LDR reading and triggers the fallback brightness.

In a real-world scenario, this phase mimics the fail-safe engineering found in advanced systems. If the SBBL's LDR sensor fails, the system ignores the unreliable data and defaults to the maximum brightness. This ensures that a hardware failure results in a safer outcome for the cyclist, where the brake lights are too bright, rather than a dangerous one, where the brake lights are too dim to be noticed.

3.5 Final System Overview and Challenges

The final Tinkercad circuit successfully integrates all five phases into a single, cohesive system. The system operates by continuously reading the four main inputs (left or right brake buttons, motion slide switch, and the manual OFF button) along with the analog LDR sensor. The Arduino processes these inputs through the state-machine logic to determine the correct operational mode (CYCLING, BRAKING, STATIONARY, or DEACTIVATED). The output is dynamically controlled using pulse-width modulation (PWN), where the state machine dictates when the three red LEDs illuminate, while the LDR dictates the LEDs' light intensity. Finally, the power management phase ensures that the microcontroller enters a low-power sleep state when idle, waking instantly when braking activity or a motion state change triggers an interrupt.

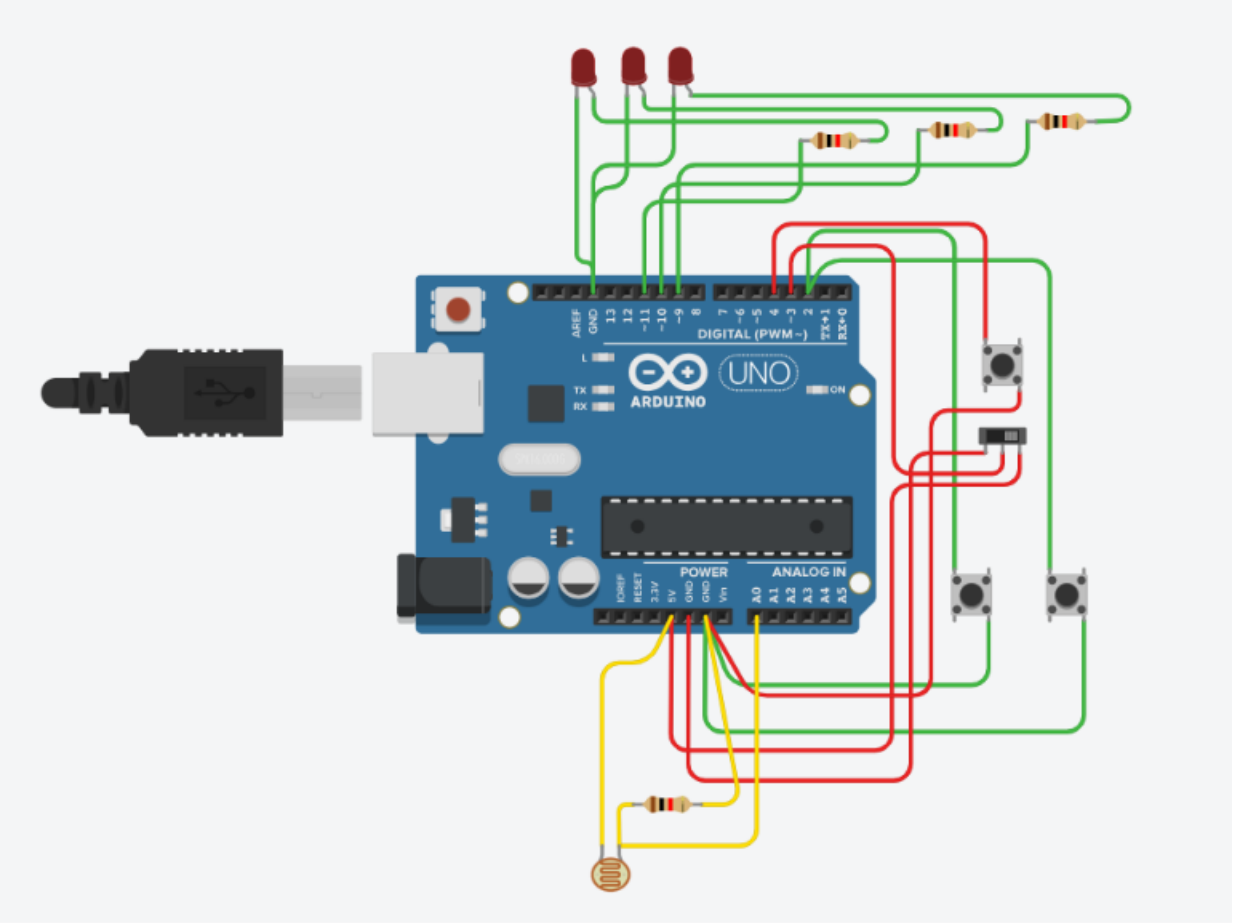


Figure 3.27 The complete and fully integrated SBBL simulation circuit.

Throughout the developmental process, several technical challenges were encountered and resolved. Firstly, the motion slide switch initially produced unreliable readings due to a floating input; this was resolved by wiring the switch to both 5V and GND to ensure it provides a clear signal. Secondly, a logical error in Phase 2 caused the system to forget that the cyclist had previously braked while the bicycle was still moving, which caused the stationary light to fail to activate. This was fixed by ensuring the system only resets its braking memory when the bicycle transitions from a stationary state back to cycling. Finally, Tinkercad's inability to simulate the microcontroller's physical power-down state meant the sleep functionality couldn't be physically measured; however, the Serial Monitor verified the logical execution of the interrupts and the sleep sequence using print statements.

Chapter 4: Testing, Evaluation, and Simulation Limitations

4.1 Testing Methodology

The testing phase was conducted to verify that the Smart Bicycle Brake Light (SBBL) system met both its functional and non-functional requirements. Since the system was developed in five stages, testing was performed in two distinct ways: modular testing and full integrated system testing.

Modular testing involved verifying each phase in isolation immediately after its development. This ensured that any errors were identified and fixed before integrating the next phase. Once all five phases were successfully combined into the final product code, integrated system testing was performed to ensure that the individual modules interacted properly without conflict.

Testing was fully conducted in the Tinkercad Circuits environment. The primary tools used for verification were the visual state of the circuit components and the Serial Monitor, which provided real-time print statements of the system's internal logic and processing.

4.2 Evaluation Against the Design Specifications

This section evaluates the final simulation against the design specifications outlined in Chapter 2. It confirms whether the proposed system architecture and the state-machine logic were successfully implemented.

Table 4.1 Component inventory for the SBBL simulation, detailing their real-world equivalents and roles in the system.

ORIGINAL REQUIREMENT	DESIGN SPECIFICATION	EVALUATION RESULT
Dual Brake Detection	Must detect the left or right brake lever.	**Pass.** Both inputs on D2 were successfully verified.
Conditional Illumination	LEDs off during movement unless braking.	**Pass.** System correctly remained off during normal movement.
Stationary Hold	Maintain illumination after stop.	**Pass.** The holdActive state successfully kept the LEDs on indefinitely.

Manual OFF Override	Allow rider to switch off stationary light.	**Pass.** Button on D4 successfully disabled the LEDs and blocked brake inputs while stationary.
Ambient Light Adjustment	Adjust PWM brightness through the LDR.	**Pass.** Successfully calibrated to 180 (Day), 218 (Dusk), 255 (Night).
LDR Fault Detection	Detect abnormal LDR reading and fall back to the maximum LED brightness	**Pass.** The 2-second threshold successfully triggered a max brightness fallback.
Power Management	Enter low-power sleep mode when idle.	**Pass.** Sleep entry and interrupt wake-up logic successfully verified using the Serial Monitor output.
Non-Blocking Operation	No delay() function in main loop.	**Pass.** millis() was used for all timing, ensuring continuous monitoring.

4.3 Functional and Non-Functional Testing Results

4.3.1 Functional Testing

4.3.1.1 Phase 1: Core Brake Detection Test Case

Table 4.2 Phase 1 Test Case

TEST ID	INPUT CONDITION	EXPECTED RESULT	ACTUAL RESULT	STATUS
P1-T1	No buttons pressed (Moving)	All LEDs OFF	All LEDs remained OFF	Pass
P1-T2	Left brake button is pressed	All LEDs ON	All LEDs are illuminated	Pass
P1-T3	Right brake button is pressed	All LEDs ON	All LEDs are illuminated	Pass
P1-T4	Both brake buttons are pressed	All LEDs ON	All LEDs are illuminated	Pass

4.3.1.2 Phase 2: Motion Recognition And Manual Override Test Case

Table 4.3 Phase 2 Test Case

TEST ID	INPUT CONDITION	EXPECTED RESULT	ACTUAL RESULT	STATUS
P2-T1	Moving, brakes released	LEDs OFF	LEDs OFF	Pass

P2-T2	Moving, brakes pressed	LEDs ON	LEDs ON	Pass
P2-T3	Brakes pressed, change switch to STOPPED	LEDs remain ON (Hold begins)	LEDs remained ON	Pass
P2-T4	Release the brakes while STOPPED	LEDs remain ON (Hold active)	LEDs remained ON	Pass
P2-T5	Press Manual OFF while STOPPED, after a braking event	LEDs turn OFF	LEDs turned OFF	Pass
P2-T6	Press brakes after Manual OFF	LEDs remain OFF	LEDs remained OFF	Pass
P2-T7	Start moving again	Hold cancelled; Manual OFF resets	LEDs OFF; system reset	Pass

4.3.1.3 Phase 3: Intelligent Lighting Management Test Case

Table 4.4 Phase 3 Test Case

TEST ID	INPUT CONDITION	EXPECTED RESULT	ACTUAL RESULT	STATUS
P3-T1	Moving, brakes released (Daylight)	LEDs OFF	LEDs OFF	Pass
P3-T2	Braking (Daylight)	LEDs ON at brightness 180 (low brightness)	LEDs ON at low brightness	Pass
P3-T3	Braking (Dusk)	LEDs ON at brightness 218 (mid brightness)	LEDs ON at mid brightness	Pass
P3-T4	Braking (Nighttime)	LEDs ON at brightness 255 (high brightness)	LEDs ON at max brightness	Pass
P3-T5	Stationary hold active (Nighttime)	LEDs ON at brightness 255	LEDs ON at max brightness	Pass
P3-T6	Manual OFF activated (Any light)	LEDs OFF	LEDs OFF	Pass

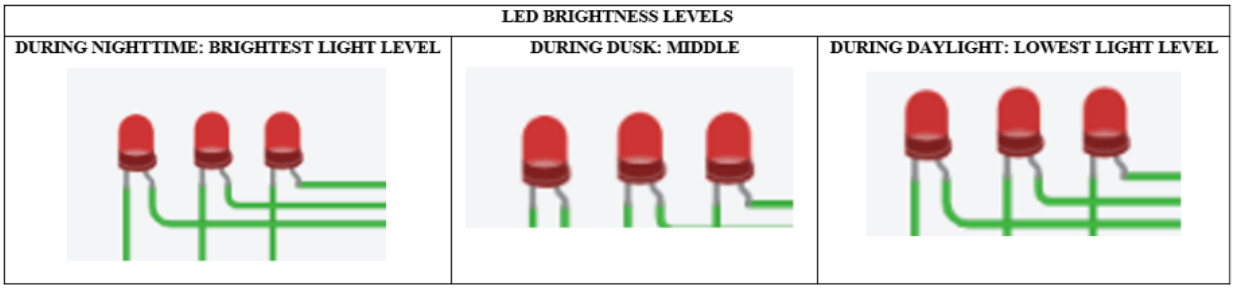


Figure 4.1 Visual comparison of the LED brightness levels. Note: Tinkercad's simulation limits the visual distinction between the pulse-width modulation (PWM) values, but the illuminations correspond to the following PWM values: Nighttime (255), Dusk (218), Daylight (180).

4.3.1.4 Phase 4: Power Management And Sleep Mode Test Case

Table 4.5 Phase 4 Test Case

TEST ID	INPUT CONDITION	EXPECTED RESULT	ACTUAL RESULT	STATUS
P4-T1	Moving, no brakes, no hold	Arduino enters sleep mode	Serial Monitor confirms sleep entry	Pass
P4-T2	Brake pressed while asleep	Arduino wakes up	Serial Monitor confirms wake trigger	Pass
P4-T3	Motion switch changed while asleep	Arduino wakes up	Serial Monitor confirms wake trigger	Pass
P4-T4	Brake pressed while asleep and Manual OFF override is active	Arduino wakes, but LEDs remain OFF	Woke up; LEDs stayed OFF	Pass

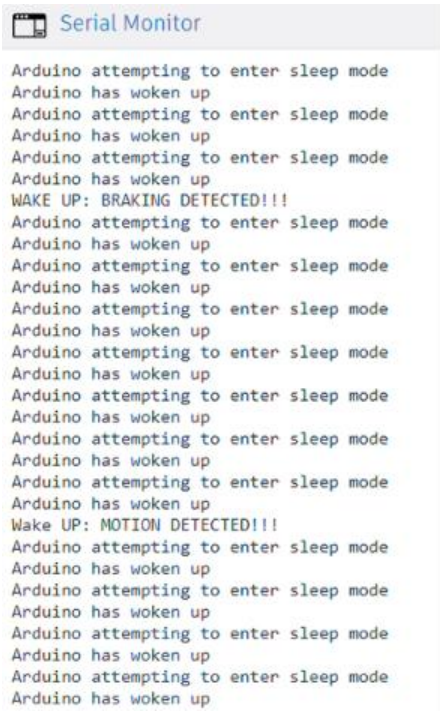


Figure 4.2 Code Excerpt: Serial Monitor output verifying the successful execution of the sleep mode and the wake-up interrupts.

4.3.1.5 Phase 5: LDR Fault Detection Test Case

(Important Note: The LDR threshold was temporarily lowered to 630 to simulate a fault, since the maximum LDR setting is 679)

Table 4.6 Phase 5 Test Case

TEST ID	INPUT CONDITION	EXPECTED RESULT	ACTUAL RESULT	STATUS
P5-T1	LDR reading normal (< 630)	Normal brightness selected	Normal brightness	Pass
P5-T2	LDR reading > 630 for 1 second	No fault flagged yet	No fault flagged	Pass
P5-T3	LDR reading > 630 for 2 or more seconds	LDRFault variable is set to true	Serial Monitor confirms fault	Pass
P5-T4	Fault active and Braking	Fallback brightness (255) is set	LEDs illuminated at max brightness	Pass

P5-T5	LDR reading returns to normal	Fault flag clears; normal brightness resumes	Serial Monitor confirms reset	Pass

4.3.2 Non-Functional Testing

Table 4.7 Non-Functional Requirements Test Case

TEST ID	REQUIREMENT TESTED	TEST METHOD	EXPECTED RESULT	STATUS
NFR-T1	Responsiveness	Observe LED reaction when a brake is pressed	LEDs activate with no noticeable delay	Pass
NFR-T2	Safety Prioritisation	Force LDR fault during braking	Braking logic overrides the LDR; LEDs stay ON at the max brightness	Pass
NFR-T3	Non-Blocking Operation	Monitor motion switch response while the hold timer is active	System responds instantly to movement changes	Pass
NFR-T4	Fault Tolerance	Simulate abnormal LDR reading	System defaults to the safe fallback brightness rather than failing	Pass
NFR-T5	Testability	Execute isolated phase tests	Components interact predictably in the simulation	Pass

4.4 Simulation Against Reality

While the Tinkercad simulation successfully verified the logical execution of the code and the interaction between the digital circuit components, it is important to differentiate between a software simulation and a physical prototype.

4.4.1 Simulation Limitations

- **Power Consumption:** Tinkercad cannot measure real power consumption. Even though the sleep logic executes, the simulator cannot prove that the microcontroller physically reduces its power consumption when set to sleep mode.

- **Component Durability:** The simulation cannot verify the durability of all the components and their wiring under real-world cycling conditions.
- **Environmental Factors:** Tinkercad cannot validate the visibility of the LEDs in the real world; it doesn't account for how the LEDs will look in actual environmental conditions, including sunlight, fog, rain, or snow. Moreover, it cannot predict how temperature changes will affect the LDR's accuracy.

In conclusion, the simulation environment has its limitations, but the test case results prove that the SBBL system's control logic, state machine, and fault detection and fail-safe mechanisms are valid and robust. Every functional and non-functional requirement that was outlined in the second chapter was successfully demonstrated in the digital environment. The system handles multiple simultaneous inputs, manages timing without blocking, and prioritises safety above everything else.

Chapter 5: Conclusion and Real-World Implementation

5.1 Introduction

The next logical phase of this project would be transforming the digital simulation into a physical hardware prototype. This would involve transitioning the circuit from a virtual breadboard to a soldered perfboard or a custom Printed Circuit Board (PCB), and uploading the certified code onto a physical microcontroller (like a compact Arduino Nano) to test real-world battery life, environmental resistance, and light visibility.

5.2 Project Conclusion

The Smart Bicycle Brake Light (SBBL) project successfully achieved all its objectives through a meticulous, five-phase modular development approach. The simulation successfully demonstrated the integration of brake detection, motion-state recognition, stationary illumination, ambient-light brightness adjustment, power management, and sensor fault detection.

By utilising a state-machine framework and non-blocking logic, the system proved that it can handle multiple simultaneous inputs, prioritise safety (ensuring the brake light is always illuminated when needed), and manage its power usage without lagging. The testing phase validated the system's robust logical execution, while the simulated environment verified the effective hardware and software integration.

5.3 Envisioned Physical Product Design

To transition the SBBL from a simulation into a commercial product, the components must be physically constructed and integrated into the bicycle. The prototype would involve replacing the Arduino Uno with a smaller microcontroller, housing all the electronics in an impermeable enclosure under the saddle, and powering the system with a rechargeable Lithium battery charged using a USB-C port.

To replicate the simulation's inputs in the real world, the virtual components would be replaced with physical sensors. Firstly, the brake detection would rely on microswitches integrated with the brake levers or mechanical cable-clamp sensors, so that when the cyclist pulls the lever, the switch closes, sending a clear signal to the microcontroller. Secondly, the simulated slide switch would be replaced with a physical motion sensor, such as a wheel-rotation sensor or an accelerometer, to detect when the bicycle stops and starts. Finally, the LDR would directly translate into the physical

product; it would simply be mounted behind a transparent window on the enclosure so that it can measure the ambient light without obstruction.

5.4 Product Models

The physical product was envisioned in two distinct configurations to accommodate different cyclists.

5.4.1 The Permanent (Pre-Installed) Model

This configuration is intended for new bicycles manufactured with the SBBL system pre-installed. This model offers maximum weather protection, cleaner aesthetics, and better security against theft.

- **Wiring:** The wiring is routed entirely through the inside of the bicycle's structure: starting from the brake lever housing, passing through the handlebars, entering the frame at the front, and travelling internally to exit near the seat post.
- **Brake Detection:** The mechanical switches are integrated directly into the internal brake-lever assemblies, providing a seamless look.
- **Power:** The battery could be integrated into the frame or housed in a permanently sealed, rechargeable unit under the saddle.

5.4.2 The Detachable (After-Market) Model

This configuration is designed as an accessible add-on that can be purchased separately and installed on existing bicycles.

- **Wiring:** External wiring is routed along the bike frame and secured with the built-in rubber closures and mounting clamps.
- **Brake Detection:** The system uses adaptable, clamp-on microswitches that attach to the existing brake cables or levers.
- **Power and Connectivity:** The battery and electronics are housed in a removable enclosure under the saddle. A waterproof quick-disconnect plug separates the rear light from the brake wiring, allowing the unit to be easily detached for charging or anti-theft purposes.

5.5 Recommendations And Future Enhancements

Even though the current simulation meets all the system requirements, several advanced features could be integrated into future configurations of the SBBL system, making it a more comprehensive smart safety system.

- **Rear-Object Detection:** An ultrasonic sensor could be added to monitor the area behind the bicycle. If a vehicle or object approaches within a dangerously close distance, the system could trigger either an audible or visible alert to warn both the cyclist and the road users behind them.
- **Bluetooth Connectivity:** Adding a Bluetooth module would allow the SBBL to pair with a smartphone app or a bike computer. This could provide real-time battery monitoring, alert the rider to system faults, and allow them to customise the brightness levels.
- **Further Fault Detection:** While the current system detects LDR faults, future versions could implement voltage monitoring to detect battery levels and current sensing to detect whether an LED has burnt out or a brake switch has disconnected, making it a fully diagnosable system.
- **Fully Integrated Smart Light System:** Future versions could merge the conventional rear lighting system with the SBBL's existing logic to create a single, comprehensive lighting unit. However, the brake and stationary illumination would remain the priority, instantly overriding the visibility modes whenever braking or stopping is detected to ensure clear signalling and maximum safety.

References

BikeRadar. (n.d.). *Lezyne Strip Drive Pro 400+ Rear light review*. From BikeRadar: https://www.bikeradar.com/reviews/accessories/lights/back-light/lezyne-strip-drive-pro-400-rear-light-review

Buehler, R., & Goel, R. (2022). Chapter Seven - A Global Overview of Cycling Trends. In *Advances in Transport Policy and Planning* (pp. 133-158). Elsevier.

Chao, C.-C., Ma, H.-P., Wei, L., Lin, Y.-N., Chen, C., Saleh, W., . . . Pai, C.-W. (2025). Risk factors for overtaking, rear-end, and door crashes involving bicycles in the United Kingdom: Revisited and reanalysed.

Hamilton, S. T., & Orsi, C. M. (2013). *Turn and Brake Signal System for a Bicycle.* San Luis Obispo: California Polytechnic State University.

JVBike. (n.d.). *Sigma Rear BRAKE Light - BLACK*. From JVBike: https://www.jvbike.com/sigma-rear-brake-light-black.html?srsltid=AYCX9vKmlnCk4ljtNUZjx_5sjzxdR45wrc_1e5wanZwwgMN7MI3YZkEF

Lezyne. (2015, December 15). *Lezyne Strip Drive | Powerful LED Bike Lights with Daytime Flash*. From Youtube: https://www.youtube.com/watch?v=wiGFXGzgVFQ

Lezyne. (2026). *Strip Drive Pro 400+ Rear*. From Lezyne: https://ride.lezyne.com/products/strip-drive-pro-400-plus-rear?srsltid=AYCX9vJPpMWEvOnixq5Ou6fSPgpd6dhIPznroizVxkhkLmpErTg5yGfb

Liao, S.-H., Lin, L. H.-M., Tsai, C.-Y., Chen, C.-C., Li, Y.-W., Huang, H.-T., & Chen, L.-S. (2025). Design of Turn and Brake Lights of Bicycles Using Advanced Wireless Transmission. *Sensors and Materials*, pp. 2279–2285.

Magene. (n.d.). *L508 Radar Tail Light*. From Magene: https://www.magene.com/en/bike-lights/55-l508-radar-tail-light.html?srsltid=AYCX9vLiwJ8gwtJtbmWfiA2qOtKHIMS2298jSUbx-XDiIvMp_kfP62kn

Pai, C.-W. (2011). Overtaking, rear-end, and door crashes involving bicycles: An empirical investigation. In *Accident Analysis & Prevention* (pp. 1228-1235). Elsevier.

Panda Podium. (n.d.). *Review: Magene L508 Radar Tail Light*. From Panda Podium: https://www.pandapodium.cc/post/review-magene-l508-radar-tail-light/

ProCylingUK. (2026, April 3). *Magene L508 radar light review: budget radar tech with pro-team credibility*. From ProCylingUK: https://procyclinguk.com/magene-l508-radar-light-review-budget-radar-tech-with-pro-team-credibility/

Sigma. (2015, August 7). *SIGMA SPORT // BRAKELIGHT // Hard Facts (EN)*. From Youtube: https://www.youtube.com/watch?v=KREkkvwmWHQ&t=4s

Stead, M. (2015, December 26). *Sigma Rear Brake Light*. From road.cc: https://road.cc/content/review/173637-sigma-rear-brake-light

Turner, A. (2025, January 8). *Lezyne Strip Drive Pro 400+ review: Fantastically bright light that holds securely and lasts well*. From CyclingNews: https://www.cyclingnews.com/reviews/lezyne-strip-drive-pro-400-review/